

Bachelor's Thesis in Computer Science

Build a Large Language Model from scratch

Giorgi Gogsadze & Revaz Gogvadze

Supervised by: Professor Walter F. Tichy

Everyone is spending more on AI

AI spend per employee per month



Source: [Ramp AI Index](#), business spend data from Ramp. AI spend includes LLM subscriptions, coding agents, API tokens, and GPU cloud spend. Percentile assignment is by per employee spend on AI. • [Get the data](#)

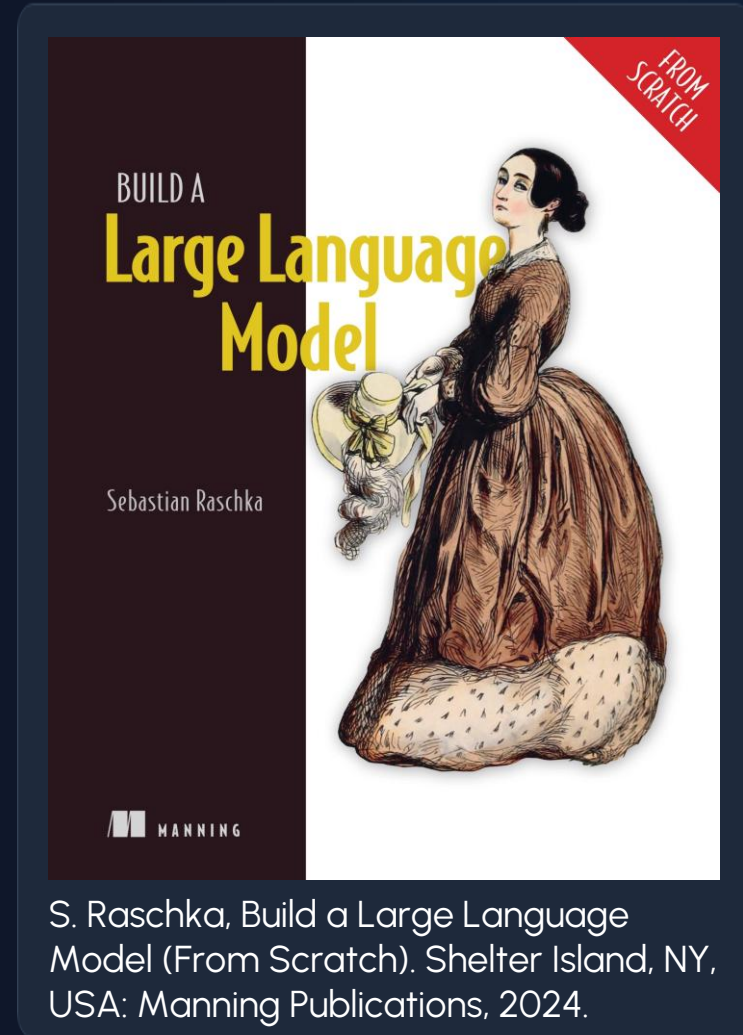
ramp 

|| When I talk to developers, CEOs, and government officials across the world, I usually hear: We need to train, fine-tune, and distill our own models. ||

— Mark Zuckerberg, Founder and CEO of Meta

| Thesis Goal

- ❓ Can a working, specialized LLM be built from scratch, or fine-tuned from a pre-trained model, without massive data centers or huge budgets?
- 🔧 We aim to test whether a local machine can realistically construct a GPT-style model.
- ☰ Train our own model and adapt public pre-trained models for real-world tasks.



S. Raschka, Build a Large Language Model (From Scratch). Shelter Island, NY, USA: Manning Publications, 2024.

Data Preparation

Byte Pair Encoding

"It is beautiful."



["It", " is", " beaut", "iful", "."]



[116, 313, 583, 659, 46]

Token Embedding

[116, 313, 583, 659, 46]

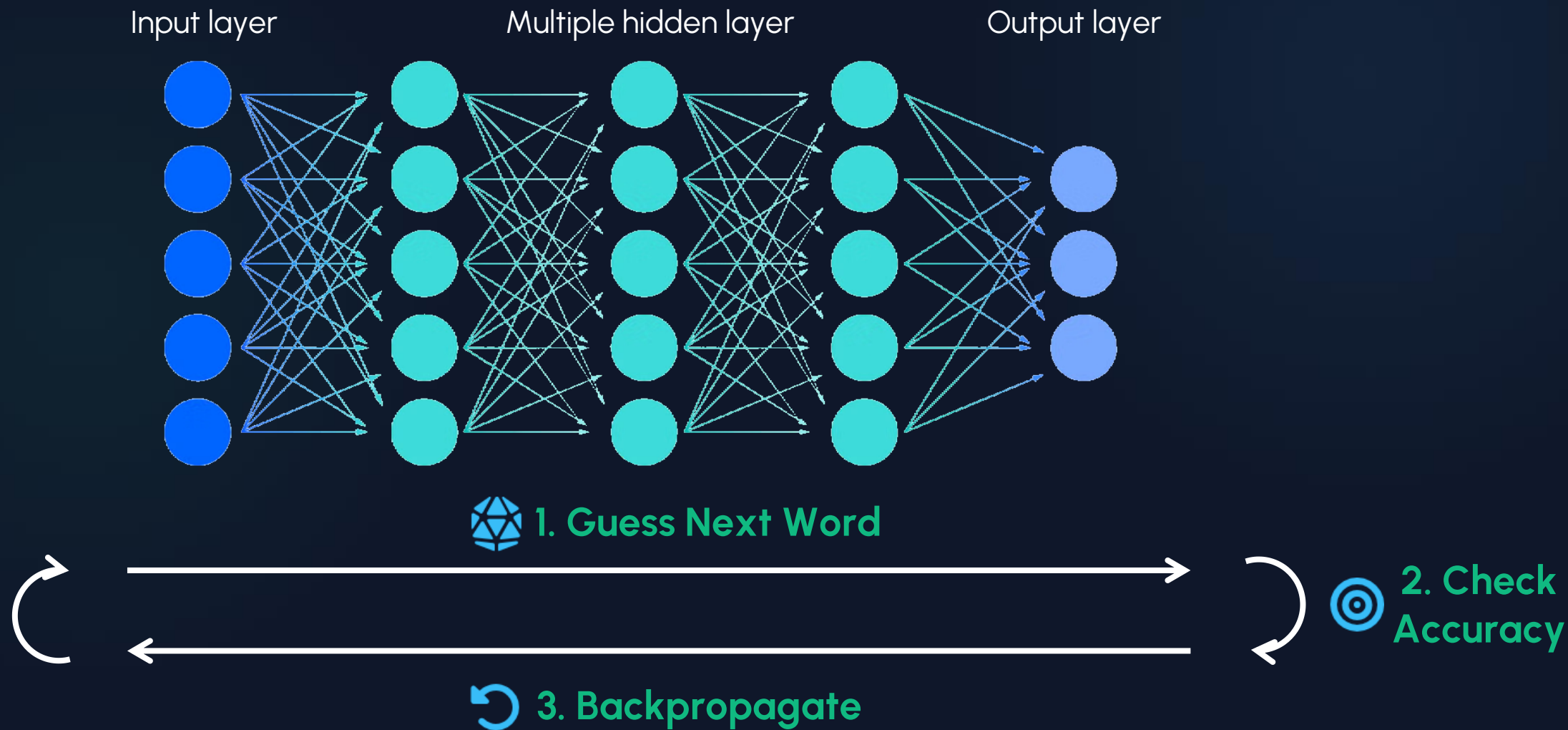


[[...], [...], [...], [...], [...]]

sliding-window data loader

```
for i in range(0, len(token_ids) - max_length, stride):  
  
    # Input sequence of length max_length  
    input_chunk = token_ids[i:i + max_length]  
  
    # Target sequence is the same sequence shifted by one token  
    # (model learns to predict the next token)  
    target_chunk = token_ids[i + 1:i + max_length + 1]  
  
    # Store tensors for PyTorch training  
    self.input_ids.append(torch.tensor(input_chunk))  
    self.target_ids.append(torch.tensor(target_chunk))
```

| How is an LLM Trained?



Multi-Head Causal Self-Attention

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_in, d_out, num_heads):
        super().__init__()
        # 1. Linear projections for Query, Key, and Value
        self.W_query = nn.Linear(d_in, d_out)
        self.W_key = nn.Linear(d_in, d_out)
        self.W_value = nn.Linear(d_in, d_out)

        # 2. Final output projection
        self.out_proj = nn.Linear(d_out, d_out)

    def forward(self, x):
        # 1. Project input sequence into Q, K, and V
        Q = self.W_query(x)
        K = self.W_key(x)
        V = self.W_value(x)

        # 2. Split dimensions into multiple attention heads
        Q, K, V = split_into_heads(Q, K, V, num_heads)

        # 3. Calculate Attention Scores (Scaled Dot-Product)
        attn_scores = (Q @ K.transpose) / sqrt(head_dim)

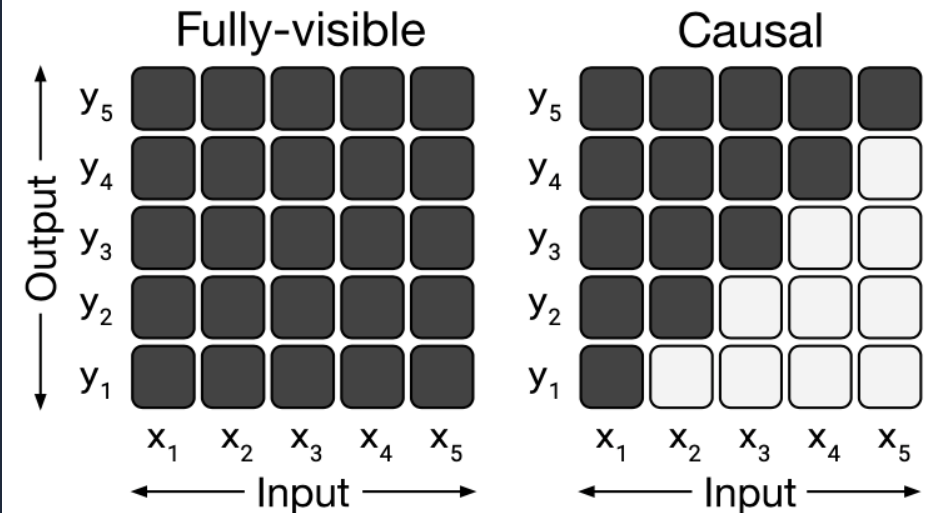
        # 4. Apply Causal Mask (prevent looking at future tokens)
        attn_scores = apply_mask(attn_scores)

        # 5. Convert to probabilities and multiply by Values
        attn_weights = torch.softmax(attn_scores, dim=-1)
        context_vec = attn_weights @ V

        # 6. Merge heads back together and project to output
        context_vec = merge_heads(context_vec)
        return self.out_proj(context_vec)
```

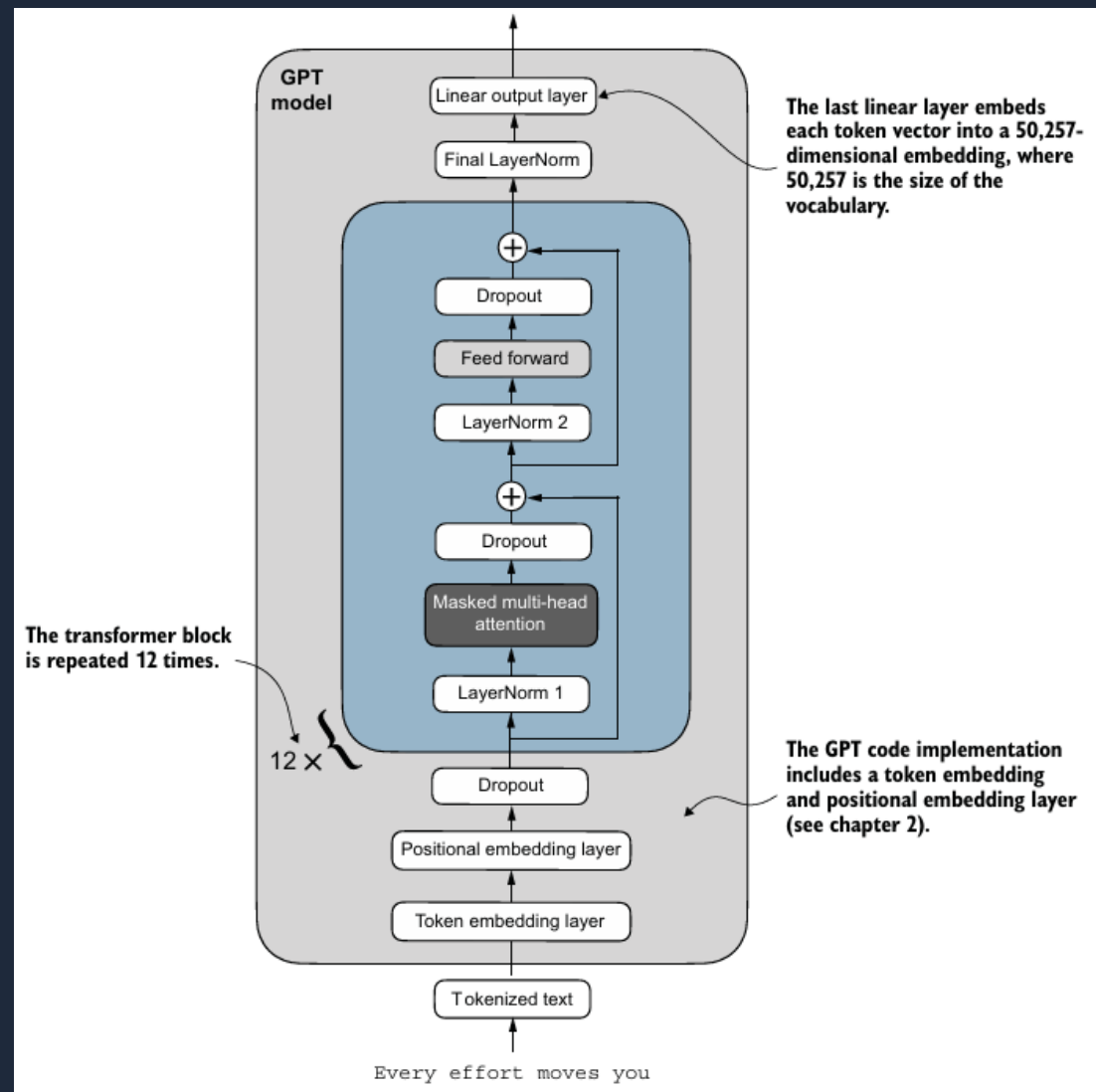
 **Multi-Head:** We split computations into multiple independent 'heads' to process different contexts in parallel.

Casual Masking



GPT Model Architecture

-  **Transformer Blocks:** We stacked 24 blocks to match the GPT-2 Medium architecture.
-  **Pre-Layer Normalization:** Added to stabilize the learning signal.
-  **GELU Activation:** Ensures smoother parameter updates within the Feed Forward network.
-  **Shortcut Connections:** Added to solve the vanishing gradient problem, letting learning signals skip layers directly.

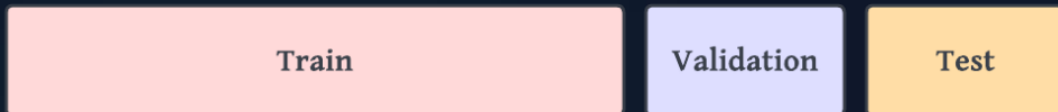


An overview of the GPT model architecture

Source: S. Raschka, Build a Large Language Model

Before Training

Data Split



The dataset is split into Training, Validation, and Test sets. The model trains on the Training set and is validated to prevent overfitting.

Cross-Entropy Loss

Measures the gap between the predicted token probability and the ground truth:

$$L = - \frac{1}{N} \sum_{i=1}^N \log P_i$$

Where **P** is the predicted probability of the true token.

Optimization: AdamW

A basic optimizer is unstable for deep transformers.

We utilized AdamW with weight decay.

$$W_{\text{new}} = W_{\text{old}} - \eta \cdot \text{gradient} - \eta\lambda W_{\text{old}}$$

W_old: Current parameter value

W_new: Updated parameter value

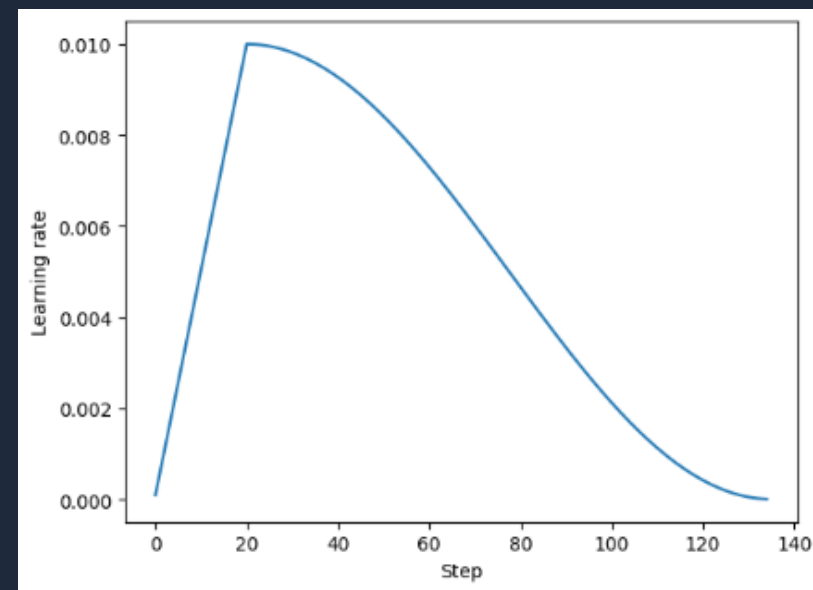
η (Eta): Learning rate

λ (Lambda): Weight decay coefficient

Warmup & Cosine Decay

↗ **Warmup:** Rate gradually increases.

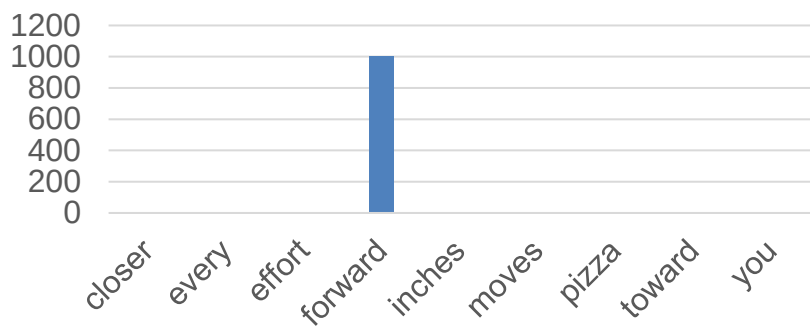
↘ **Cosine Decay:** Smooth, curved reduction for stable convergence at the end of training.



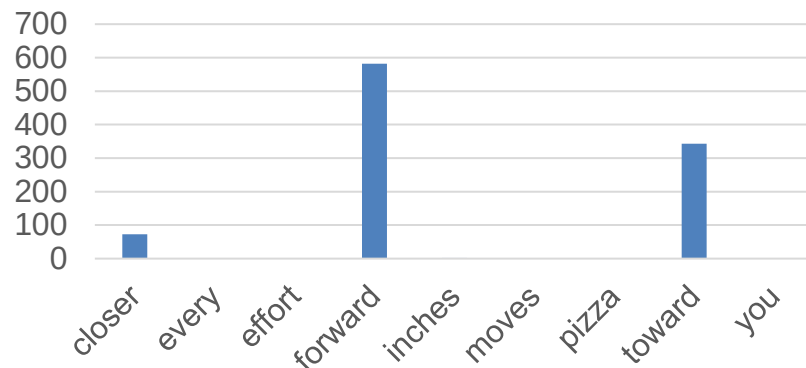
Temperature & Top-K Experiment

Give the model input "Every effort moves you" and run it 1000 times with different temperatures

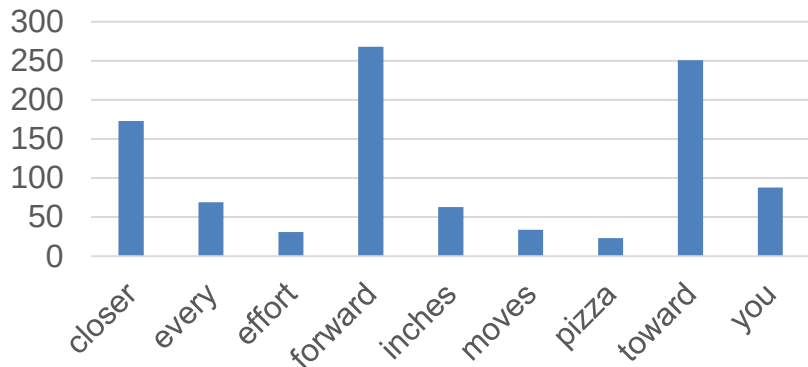
Greedy Decoding
temperature 0



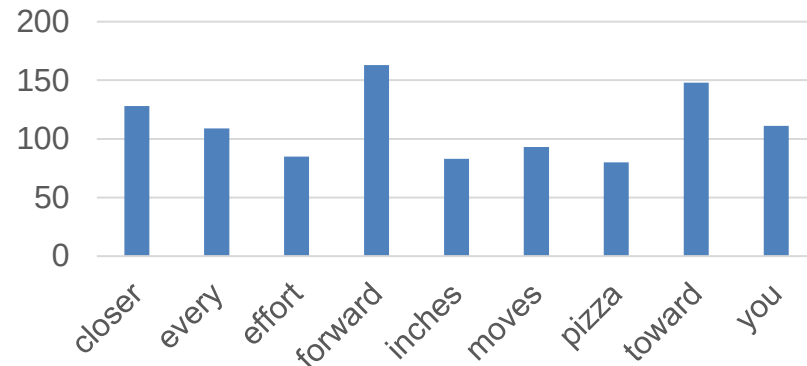
Temperature 1



Temperature 4



Temperature 16



FINE-TUNING PROJECT 1

Sentiment Classification

Adapting a generative model to predict sentiment of the sentence

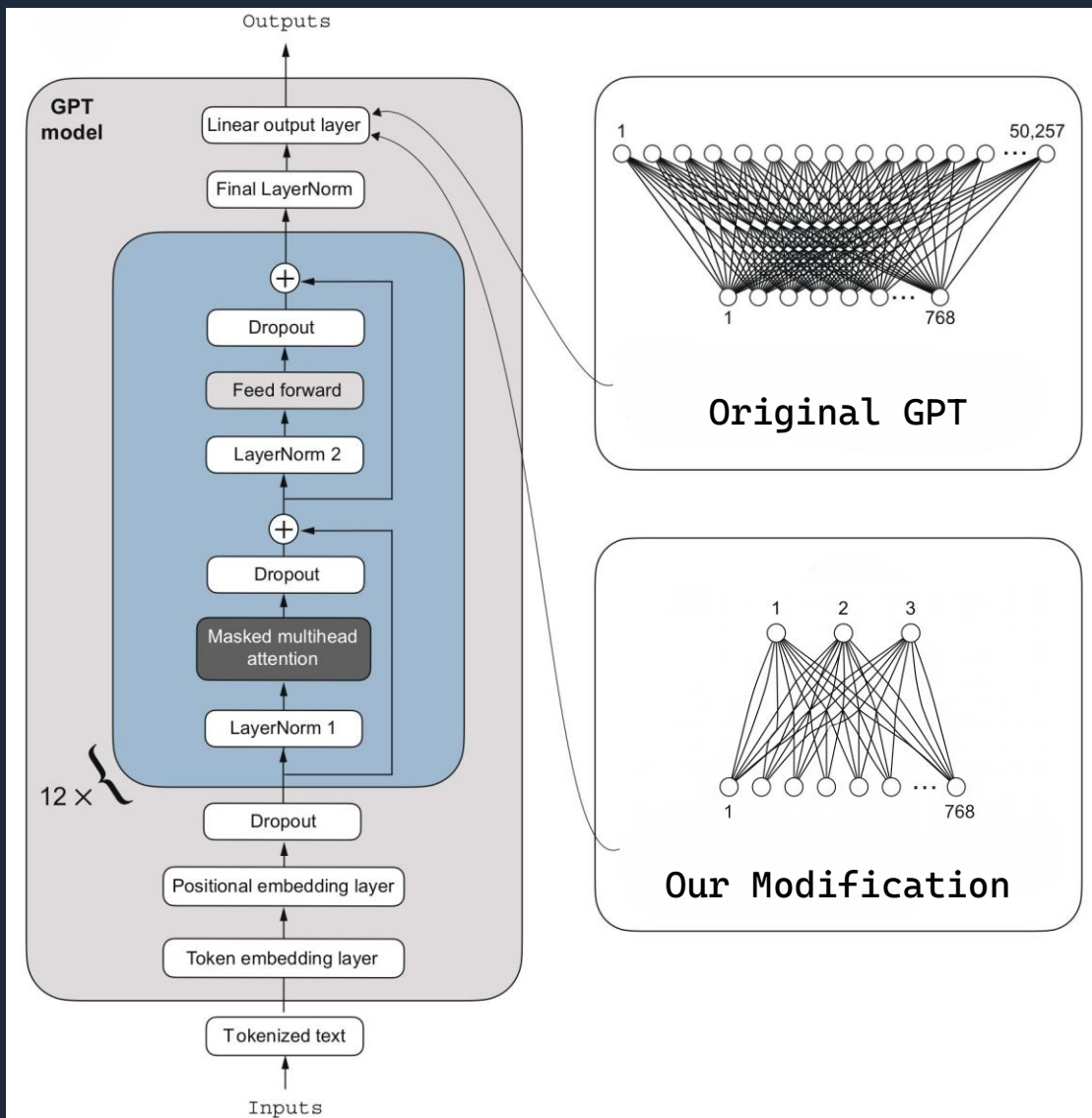
Positive

Negative

Neutral

	Label	Text
0	neutral	I'd have responded, if I were going
1	negative	Sooo SAD I will miss you here in San Diego!!!
2	negative	my boss is bullying me...
...	...	...
31013	positive	_sutra what is your next youtube video gonna b...
31014	positive	http://twitpic.com/4woj2 - omgssh ang cute n...

Architectural Modification

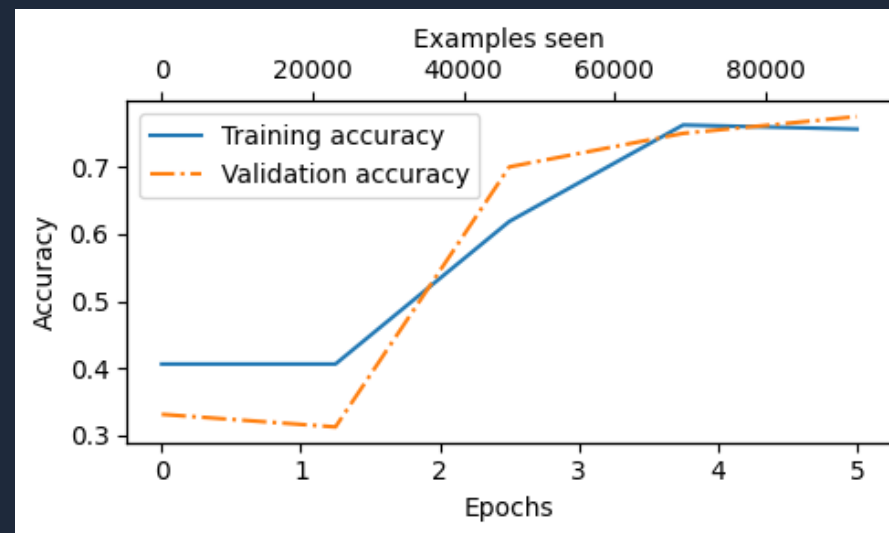
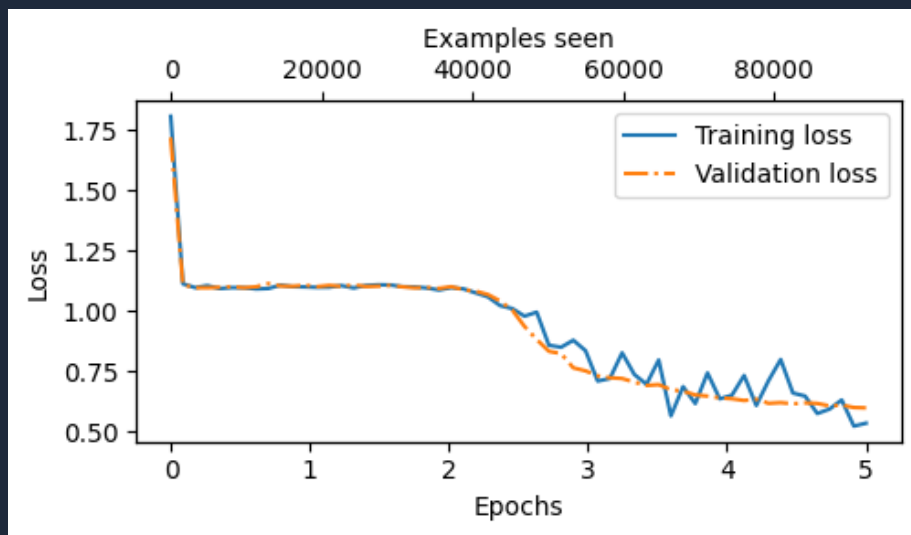


❄ Freeze Layers for Efficiency

To train locally, we froze layers that already understood general language and left 3 for training:

- ✓ 12th Transformer Block
- ✓ Final Layer Normalization
- ✓ New Classification Head

Quantitative Evaluation



23

Minutes to Train Locally

By leveraging layer freezing on a local setup, the model transitioned from random guessing to ~75% accuracy with minimal overfitting in under half an hour.

Qualitative Evaluation

Input Sentence	Model Output	Accuracy
I did amazing in the exam today. Finally, I'll get good grades.	Positive	✓ Correct
Today I got so tired, whole day I'm working like a dog...	Negative	✓ Correct
The parliament of Georgia elected new Prime minister today.	Neutral	✓ Correct
today was my first day at the university. It wasn't really good!	Negative	✓ Correct
I can't say this wasn't one of the best experiences I've had.	Positive	✓ Correct
I'm not unhappy with the results. quite the opposite.	Negative	✗ Incorrect

Academic Writing Assistant

Sequential instruction fine-tuning to improve the sentence

Grammar

Instruction	Input	Output
Correct the grammar of the text.	What gets for Karnal pincode?	How to get Karnal pincode?

Naturalness

Instruction	Input	Output
Rewrite the text to sound more natural and fluent while preserving meaning.	When discussing the pros and cons of changing the school menu	The best way to provides healthy food for student is change

Academic Styling

Instruction	Input	Output
Rewrite the text in polished academic English.	Wow, I am very dumb in my observation skills	I do not have good observation skills

Optimization via LoRA

Low-Rank Adaptation (LoRA) trains only small additional matrices A and B while keeping the base model completely frozen.

$$\Delta W \approx AB$$

$$W_{\text{new}} = W + \alpha r \Delta W$$

```
class LoRALinear(nn.Module):
    def __init__(self, linear_layer, rank=8, alpha=16):
        super().__init__()
        self.linear = linear_layer # The original, frozen layer

        in_features = linear_layer.in_features
        out_features = linear_layer.out_features
        device = linear_layer.weight.device

        # Trainable LoRA matrices
        self.lora_A = nn.Parameter(torch.empty(in_features, rank, device=device))
        self.lora_B = nn.Parameter(torch.empty(rank, out_features, device=device))
        self.scaling = alpha / rank

        # Standard initialization
        nn.init.kaiming_uniform_(self.lora_A, a=math.sqrt(5))
        nn.init.zeros_(self.lora_B)

    def forward(self, x):
        # Base forward pass (frozen) + LoRA forward pass (trainable)
        return self.linear(x) + (x @ self.lora_A @ self.lora_B) * self.scaling
```


Sequential Fine-Tuning Strategy

Run	Initialized from	Runtime (min)	Train loss	Validation loss
 Stage 1 Grammar	base gpt2-medium	17.20	1.5376	1.5019
 Stage 2 Naturalness	Stage 1 final checkpoint	18.57	2.2689	2.1900
 Stage 3 Academic	Stage 2 final checkpoint	24.88	1.2897	1.2408

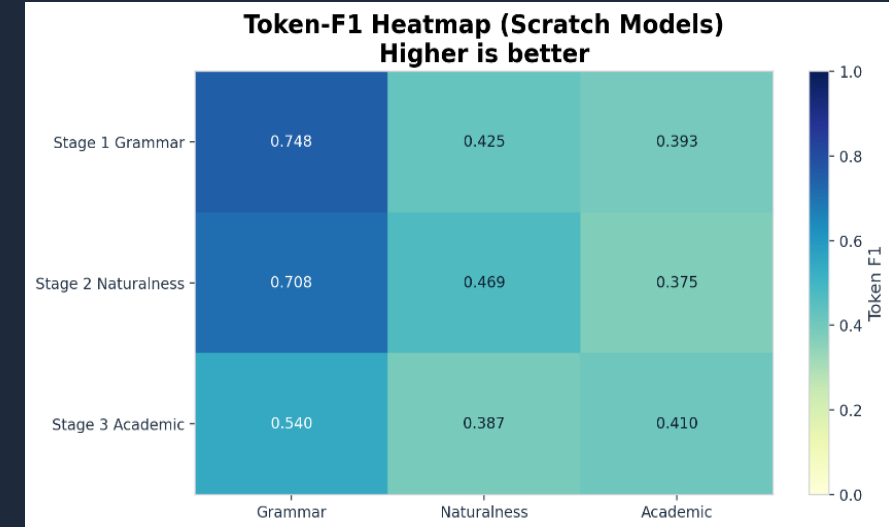
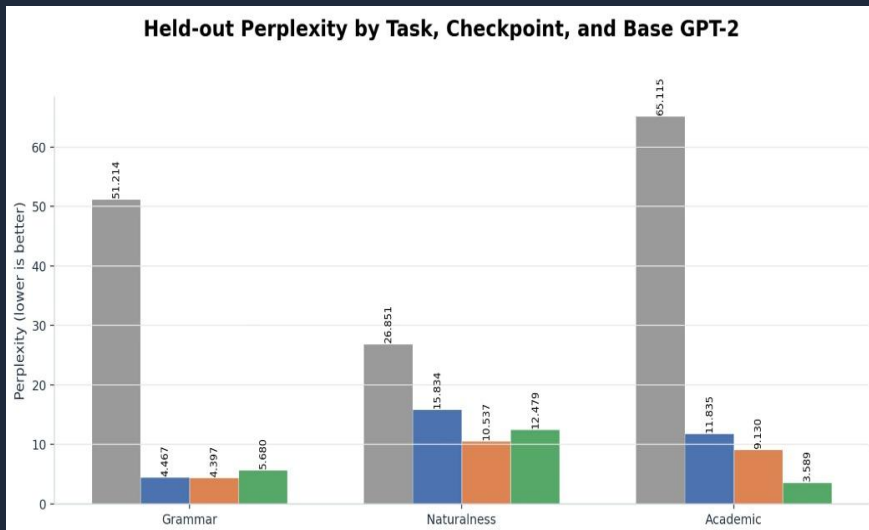
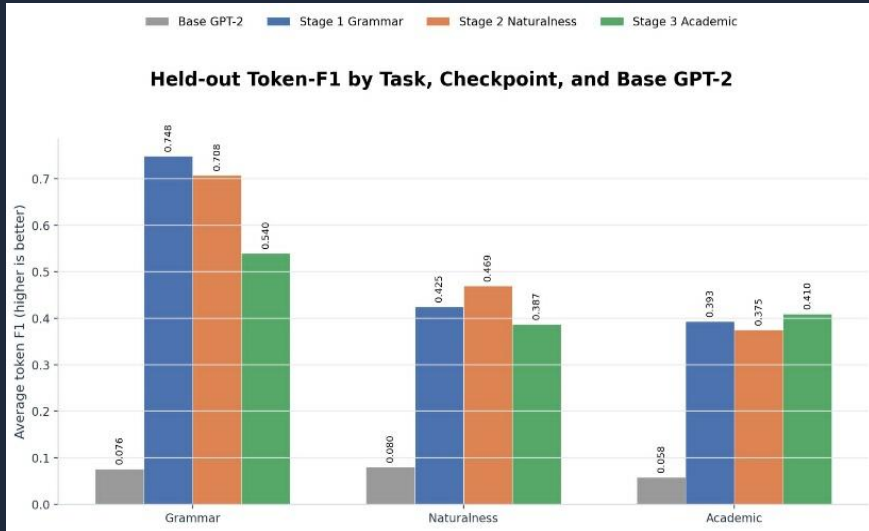


Efficiency Milestone: Total Pipeline Runtime: **61 Minutes**

Executed on a Modal L40S GPU instance.

Quantitative Results

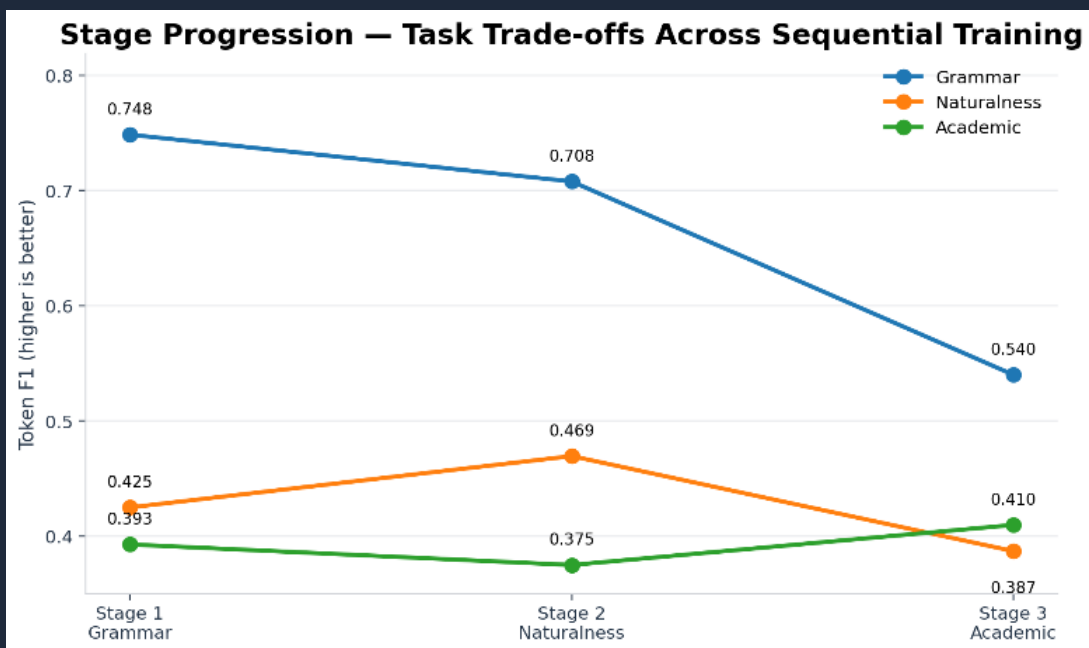
 **F1-Score Growth:** Clear improvements over base GPT-2 model.



 **Task Specialization:** Heatmap confirmed each checkpoint peaks on the task it was trained for

 **Data Sensitivity:** Perplexity dropped significantly across every stage.

Catastrophic Forgetting



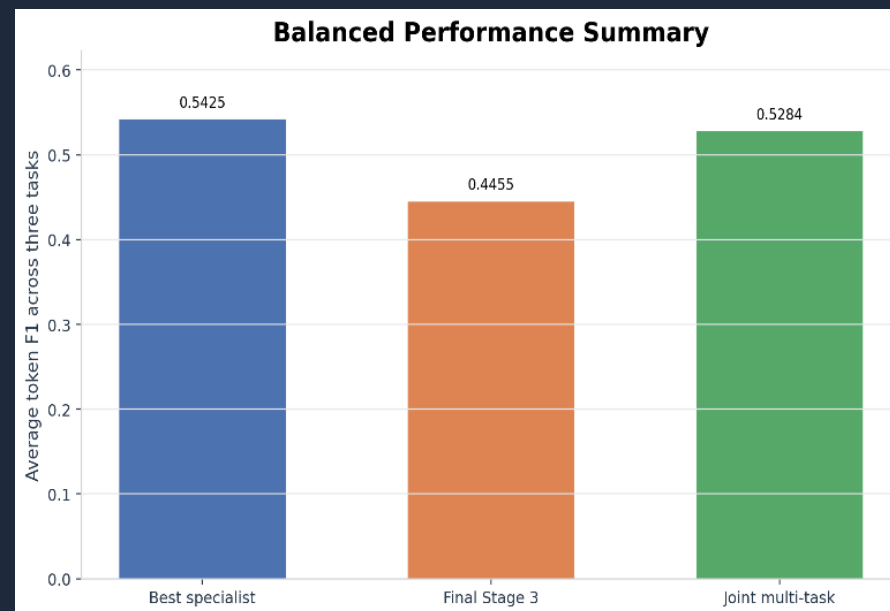
Task Trade-offs

Sequential training revealed a flaw: later stages overwrote earlier skills.

↗ Each checkpoint peaks purely on the task it was actively trained for.

↓ Adapting toward academic writing dropped strict grammar accuracy.

Joint vs. Sequential Optimization



Joint Multi-task

Dataset combined into a single training run.
Slightly higher **Average Accuracy** across all tasks compared to the final Stage 3 checkpoint.

Sequential Method

Achieved superior results on **individual specialized tasks**. Trade-off: Vulnerable to "Catastrophic Forgetting" of earlier stages.

Core Discovery: Our performance ceiling was determined by **dataset quality and hardware limits**, not the sequential design itself.

Qualitative Evaluation

Instruction	Input	Sequential Model Output
Correct the grammar of the text.	I am used to wake up early.	I am used to waking up early.

Instruction	Input	Sequential Model Output
Rewrite the text to sound more natural and fluent while preserving meaning.	Finally, the children started singing. The children danced vividly too	The children started singing and dancing.

Instruction	Input	Sequential Model Output
Rewrite the text in polished academic English.	the numbers show that our approach is the best	the results showed that the approach we used is the best

Conclusions



Local Feasibility

A functional, highly specialized LLM can be successfully built and adapted on modest, local hardware resources.



High Efficiency

Methods like layer freezing and LoRA adapters save massive amounts of compute while costing very little in accuracy.



Data Quality

The ultimate ceiling for local LLMs is dataset quality, not the architectural limits or sequential design itself.

|| The future of AI is not one single gigantic model that does everything, but millions of smaller, specialized, open-source models running locally. ||

— Clément Delangue, CEO of Hugging Face

Thank you for your attention!

We are now ready to answer your questions

All our code and experiments are openly available on GitHub:

